

NPSC3000 End of Year Report: Can Reinforcement Learning Outperform Traditional Methods for Cloud Resource Allocation

Student: Ty Riekert

Supervisor: Elham Mardaneh

Co-supervisor: Tony Mathew

Sept 2026

Abstract

The problem of resource allocation (RA) is a critical issue amongst cloud providers [1, 2, 3, 4, 5]. Consider a shared pool of physical servers that must serve a continuous stream of user jobs while meeting a variety of metrics like meeting deadlines, reducing tardiness and limiting energy usage. Classical scheduling methods are known to struggle with the dynamic, multi-objective nature of this problem [6, 7, 8], while reinforcement learning (RL) has held promising results in similar settings [1, 9, 6, 10]. This project asks **under which conditions and objectives does RL outperform classical scheduling methods**. We model this cloud RA problem as a resource-constrained scheduling optimisation problem in two settings: offline, where all jobs are known in advance, and online, where jobs arrive randomly throughout some time period. We consider a multitude of RL design decisions on proximal policy optimisation (PPO) and advantage actor-critic (A2C) RL algorithms against 49 heuristic combinations and the CP-SAT exact solver. On the main objective (Eq. (4)), optimising for only weighted squared tardiness, the best RL agent is 5% behind the best heuristic offline. When the objective also counts linear tardiness or the number of late jobs, RL outperforms the best heuristic by 2–10% offline. Online, RL is more sensitive to training budget and seed: increasing training from 300k to 1M steps improved results substantially, and the best seed at 1M steps comes within 4% of the best heuristic, although the mean over seeds remains 21% behind. Overall, RL is most competitive when the objective combines several metrics, but struggles in the online scenario, where training increases in complexity substantially.

1 Project Aims and Deliverables

The aim of this project is to formulate cloud resource allocation as a mathematical optimisation and reinforcement learning problem, and to implement and rigorously evaluate RL models against established methods. The specific deliverables are:

- Conduct comprehensive research on current findings and machine learning in cloud resource allocation
- Formulate a mathematical model of the cloud RA problem, expressed as a resource-constrained scheduling optimisation problem
- Create a corresponding Markov Decision Process (MDP) formulation for the reinforcement learning agent
- Implement the code of the scheduling environment
- Implement the code for the reinforcement learning agent(s)
- Evaluate the trained agents against classical scheduling heuristics to determine under what conditions reinforcement learning outperforms traditional methods.

The last item is not done with a single ‘train and compare’ run, but is an iterative process repeated throughout the second half of the project: a constant loop of training, evaluating against baselines, examining and researching behaviour and faults, and revising.

2 Background

2.1 Cloud Computing

Cloud computing refers to the use of computing services and resources delivered over the internet, and resource allocation (RA) is one of the most critical challenges facing cloud providers today [1, 11]. As cloud infrastructure underpins the majority of modern computing services, effectively allocating physical resources, like compute or network bandwidth, to a continuously arriving stream of workloads has become of vital importance to providers [2]. Poor allocation leads to poor metrics in response time, environmental sustainability, energy or power efficiency, throughput, makespan, utilisation and more [12, 13, 5, 14].

2.1.1 Cloud RA

Cloud RA can be decomposed into several coupled problems. Virtual machine (VM) placement decides which physical server hosts each virtual machine, subject to resource constraints like CPU and memory. This is widely regarded as the core RA subproblem [1]. Task scheduling decides when and on which resource each user task should run, and considers metrics such as execution time, response time or deadline of work [6]. Autoscaling dynamically adjusts resource capacity to match fluctuating demand in real time [9], and load balancing distributes incoming tasks evenly to avoid hotspots [15].

Solutions to these problems generally fall into three categories:

- **Exact Methods.** Algorithms that always solve a problem optimally.
- **Approximation Methods.** These methods are used when exact algorithms become computationally infeasible. Two main classes are heuristics, problem-specific rules designed to quickly construct or improve a single solution, and metaheuristics, general-purpose strategies that can guide heuristic methods [16].
- **Machine learning.** Methods that learn decision rules from data instead of manually from people.

Exact methods guarantee optimality on small instances, scale terribly, and in practice are limited to providing best-solution bounds. Heuristics and metaheuristics scale well, but can yield approximate solutions that may not be sufficiently accurate. Systematic reviews within this field identify five major problems of these approximation methods: scalability, adaptability, multi-objective complexity, high computational cost with scale, and real-time decision making [5, 6, 7, 8, 17, 18].

2.1.2 Motivations for Machine Learning

The above-mentioned limitations motivate a growing body of work applying machine learning, and reinforcement learning specifically, to combinatorial and scheduling problems. A fixed heuristic relies on designer intuition to determine what makes a job important and applies that rule unchanged through deployment. RL still requires design choices, such as what it should optimise for, but the policy for assigning jobs to machines is learnt during training, in an environment that should reflect the deployment environment [17]. Systematic reviews of RL for scheduling or bin packing report encouraging growth in this area [8], and prior systems like DeepRM [19] and Decima [20] showed learned schedulers matching or exceeding hand-designed rules on cluster workloads.

This project examines two actor-critic RL methods, Proximal Policy Optimisation (PPO) and Advantage Actor-Critic (A2C), the synchronous version of A3C [21, 22, 23, 24, 25, 26]. We also examine variations like hyper-heuristics [27, 28, 29] and hybrid RL designs [30, 31]. These learning algorithms are then placed against classical dispatching heuristics [32], a metaheuristic and an exact solver [33].

2.1.3 Project Motivation

RL for cloud RA is in active research [17, 8], but existing papers mainly review only a few objectives for RL, and compare against a limited set of baselines. Most model the problem as bin packing or VM placement, and few focus on a multi-objective resource-constrained scheduling view [34]. We would like to see:

Under which objectives and conditions does RL outperform classical scheduling algorithms on the problem of cloud RA?

2.1.4 Markov Decision Process

The Markov Decision Process (MDP) is a foundational framework for decision making in stochastic and uncertain environments [35]. It is the core structure used for an RL program. At any time t it has a state s_t that reflects the current state of the environment, and the actions available a_t (often masked to get rid of infeasible actions [36]). The agent chooses an action a_t , and the environment moves from state s_t to s_{t+1} .

An MDP is described as a tuple (S, A, P, R, γ) . We have the sets S (state space) and A (action space). P is the transition probability that satisfies the Markov property,

$$p(s_{t+1} \mid s_1, a_1, s_2, a_2, \dots, s_t, a_t) = p(s_{t+1} \mid s_t, a_t). \quad (1)$$

$p(s' \mid s, a) \in P$ describes the probability of transferring to a specific state s' by taking action a in the given state s . The reward function $R : S \times A \rightarrow \mathbb{R}$ gives the expected immediate reward $R(s, a) = \mathbb{E}[r_t \mid s_t = s, a_t = a]$ that the agent receives after taking action a in state s . Finally, $\gamma \in (0, 1]$ is the discount factor that can be used to indicate short-term or long-term importance of rewards. The MDP loop can be seen in Fig. 1. The final goal of an agent in an MDP is to maximise the overall reward.

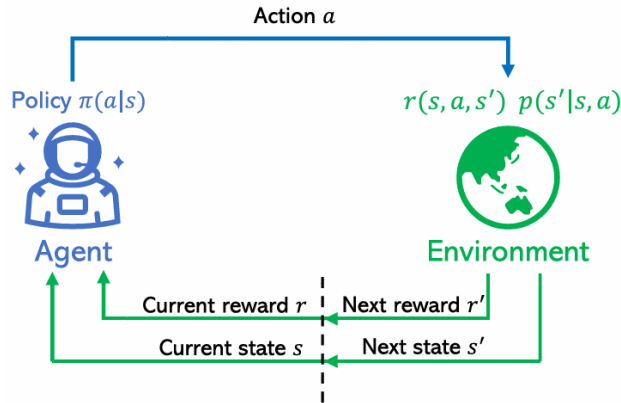


Figure 1: Agent Interaction With MDP

2.2 Reinforcement Learning

RL is a decision-making computer program that starts with no prior knowledge, and learns to make decisions from repeated interactions with an environment, using the reward for its decisions as feedback [37]. The problem is formalised as a Markov Decision Process (MDP) (Section 2.1.4) [37, 35]. The agent learns a policy: a probability distribution over actions a_t for a given state s_t , chosen to maximise the expected total reward.

RL methods fall into two main families. **Value-based** methods learn the expected return of each action and greedily choose the action with the highest return. **Policy-gradient** methods directly adjust the policy parameters towards a higher return [38]. **Actor-critic** methods combine the two: a critic (a value function) learns the expected return from each state, and the actor (a policy-gradient method) uses the critic’s estimates to judge how good a decision was and guide its updating of the policy [26].

3 Methodology

3.1 Problem Formulation

A cloud computing environment receives a set of jobs that need to be assigned to a set of physical machines. Each job j has a fixed processing duration p_j , a fixed multi-dimensional vector of resource requirements a_{jr} (e.g. CPU, memory) which it occupies for its entire duration, and a due date d_j by which the job should be completed. Jobs are assumed to be independent of one another. To reflect that some jobs matter more than others, each job is given a priority weight w_j , which we assume is set externally according to business needs.

Jobs are served by a set of identical machines M , each limited only by a fixed capacity C_r . A machine may process several jobs simultaneously, provided their combined resource usage does not exceed its capacity in any dimension. We model the scheduling to be non-preemptive: each job is assigned to exactly one machine at a single start time, and runs uninterrupted until completion. We discretise time into integer steps over a finite planning horizon H . The tardiness of a job j is then

$$T_j = \max(0, s_j + p_j - d_j), \quad (2)$$

and $U_j = 1$ if job j finishes late. Notation is summarised in Appendix A, and the full constraint set in Appendix B.

3.2 Objective Function

The first objective considered the number of machines used, weighted tardiness, and peak utilisation θ ,

$$\min \lambda_1 \sum_{m \in M} y_m + \lambda_2 \sum_{j \in J} w_j T_j + \lambda_3 \theta. \quad (3)$$

Experiments under Eq. (3) found it could be gamed and hacked [39]: unscheduled jobs incurred no loss, the machine activation penalty provided no real input (a once-off penalty), and the peak-utilisation term worked against energy-efficient consolidation [40, 41]. It was replaced by a weighted sum of optional metrics:

$$\min J_\lambda = \underbrace{\sum_j w_j T_j^2}_J + \lambda_T \sum_j w_j T_j + \lambda_U \sum_j w_j U_j + \lambda_E E, \quad (4)$$

covering squared tardiness, linear tardiness, the weighted number of late jobs, and energy (measured as total active machine-time [41]), respectively. The main objective of this project was squared tardiness alone (J), penalising a few very late jobs more than many slightly late jobs. At the end of the scheduling horizon, instead of ending the episode, jobs continue to be placed and accumulate tardiness until completion. This way, leaving a job unscheduled is always costly, removing the loophole the original objective had. The other metrics in this new objective are only considered in the multi-objective experiments.

3.3 Offline and Online Cases

We would like to study two cases of the problem: offline, when all jobs are known and available at $t = 0$ but only a single job can be assigned at each time index, and online, where jobs arrive (are revealed) over time through a Poisson process [42, 19] but multiple jobs can be assigned at any given time step.

The difficulty of the online case is set by the load factor ρ , which measures how busy the cluster is on average. A job j brings $p_j a_{jr}$ units of work. Jobs arrive at rate ν per time step, so on average $\nu \mathbb{E}[p_j a_{jr}]$ units of work arrive per time step, while the cluster can process at most $|M| C_r$ units per time step. Their ratio is the utilisation, and taking the largest ratio over the resources gives the bottleneck,

$$\rho = \max_{r \in R} \frac{\nu \mathbb{E}[p_j a_{jr}]}{|M| C_r}. \quad (5)$$

Work consequently accumulates when $\rho \geq 1$. Loads from $\rho = 0.50$ to 1.10 are studied, with log-normal job sizes matching the right-skewed distributions of real cluster traces [43, 44].

3.4 Markov Decision Process

To formulate this problem for our algorithms, we use a Markov Decision Process (S, A, P, R, γ) as discussed in Section 2.1.4.

State S . We define the state to hold each currently waiting job's ($j \in J_t \subseteq J$) features $F_j = (p_j, a_j, d_j, w_j)$, along with each currently processing job, where a job enters the processing set $P_t \subseteq J$ when it gets assigned. For each processing job, we instead hold $G_j = (m_j, s_j, p_j)$: which machine the job is assigned to, its start time and its total duration. On top of the jobs, we include the remaining capacity of each machine,

$$R_{mrt} = C_r - \sum_{j \in P_t: m_j=m} a_{jr}, \quad (6)$$

and $y = (y_m)_{m \in M}$ to record which machines are switched on. This yields a state vector,

$$s_t = (J_t, P_t, \{F_j\}_{j \in J_t}, \{G_j\}_{j \in P_t}, \{R_{mrt}\}_{m \in M, r \in R}, y, t). \quad (7)$$

Action space A . At each step, the agent can assign waiting jobs to machines, and/or idle,

$$A_t = \{(j, m) : j \in J_t, m \in M\} \cup \{\text{idle}\}. \quad (8)$$

For the offline case, an agent can only produce a single action at each time step. For the online case, an agent can continuously take actions at the same step t , only progressing to the next time step when it decides to idle.

Transition dynamics P . An assignment action (j, m) moves job j from J_t to P_t with $s_j = t$ and $m_j = m$, activating machine m if needed. When time advances (after every action offline, and only after an idle online), completed jobs leave the processing set by Eq. (9), their capacity is released, and online, newly arrived jobs join J_{t+1} . Transitions are deterministic offline, and stochastic online only through arrivals.

$$P_{t+1} = \{k \in P_t : s_k + p_k > t + 1\}. \quad (9)$$

Reward R . The reward is the negative of J , charged as tardiness accrues, plus a potential-based shaping term that provably leaves the optimal policy unchanged [45].

3.5 Methods Compared

In this project, a variety of algorithms are examined, grouped into four categories. We summarise these algorithms as follows.

3.5.1 Learning Algorithms

Reinforcement Learning. All RL policy methods examined are actor-critic methods: A2C and PPO, chosen for their robustness and simplicity. We consider these algorithms under several action-space designs:

- **Option 0 – Direct Job-Machine Selection:** The policy picks a (job, machine) pair with a pointer-network architecture [46, 47]. A large discrete action space is a known struggle for policy-gradient methods.
- **Option 1 – Hyper-heuristic Rule Selection:** The policy chooses which dispatching rule to apply at each decision [27, 28].
- **Option 2 – Priority Learning:** The policy picks which job to start next, learning its own priority from the job features; a fixed First-Fit rule then places it, making this a hybrid of RL and a dispatching heuristic.
- **Option 3 – Priority Learning with ATC:** Option 2 with the ATC index [32] as an extra feature; Option 3w is restricted to the 20 most urgent jobs [19].
- **Option 4 – Action Branching:** The job and machine are chosen by two separate heads instead of together (flat) [48].

Moreover, variants of the two RL algorithms, Reward Constrained Policy Optimisation (RCPO) for a different reward function approach, and PPO-Lagrangian, were also examined.

3.5.2 Baseline Algorithms

Dispatching Heuristics. Each heuristic combines a priority rule to choose the next job, and a placement rule to choose the next machine.

- **Priority Rules:** Earliest Deadline First (EDF) and Least Slack Time (LST), which prioritise urgency; Shortest and Longest Processing Time (SPT, LPT); Weighted SPT (WSPT) [49]; First-Come-First-Served (FCFS, also known as FIFO); the Apparent Tardiness Cost (ATC) rule, balancing job weight, length and urgency [32]; WMDD [50] and COVERT [51]. Three weighted rules were also built for this project after the first RL results: WLST and WEDF, which scale a job’s slack by its weight, and MDC, which applies Smith’s ratio rule [49] to the marginal increase in squared tardiness.
- **Placement Rules:** First-Fit, Best-Fit, Worst-Fit, and Consolidate, a more energy-aware approach [40].
- **Tetris:** A multi-resource packing heuristic scoring job-machine pairs jointly [52].

This gives 49 heuristics in total.

Metaheuristic. Particle Swarm Optimisation (PSO) [53], in which each particle encodes a job ordering decoded into a schedule, was tested earlier in the project but was always worse than the best rule, so it is not part of the final comparison.

Exact Solver. CP-SAT [33], a constraint-programming solver, models the scheduling problem exactly. Given 60 seconds per instance, it returns the best schedule it has found, an upper bound on the optimum, together with a proven lower bound.

3.6 Implementation

The environment, agents and evaluation pipeline are implemented in Python¹ using Gymnasium and Stable-Baselines3 [54]. The original environment and mathematical formulation were implemented in code manually, with all model implementations, training and evaluation done with Claude Code (see the Generative AI Use Statement).

3.6.1 Experimental Setup

Offline instances have 100 jobs, 10 machines, 4 resources with $C_r = 30$, $H = 100$. Job durations and demands are drawn uniformly from $\{1, \dots, 9\}$, and job weights from $\{1, \dots, 5\}$, with deadlines of medium tightness ($TF = 0.5$, where TF is the tardiness factor affecting deadline tightness) [55]. A matching preset with all $w_j = 1$ is generated to isolate the effects of considering weights.

Online instances use the same setup (always with weighted jobs), with the addition of Poisson arrivals over $[0, H)$ at loads $\rho \in \{0.50, 0.75, 0.95, 1.10\}$ and deadline slack drawn between 10 and 60. Moreover, job resource sizes are log-normal, mean-matched to the offline distribution. RL was trained and evaluated only offline at $TF = 0.5$ and online at $\rho = 0.95$; the other settings were run for heuristics only.

¹<https://github.com/Ethan-Ty-Riekert/Neural-Knapsack-Research-Project>

Training and tuning. An agent learns over episodes, each a complete, newly generated problem instance, so it never trains on the same problem twice. Training, validation (20 instances) and test (50 instances) use disjoint random seeds, so test instances are never seen in training or tuning. Every combination of design (6), algorithm (2) and case (2) had its own hyperparameter search, 24 studies in all, run with the same protocol: Optuna [56] with the TPE sampler [57], 12 trials of 300,000 steps, the first trial always being the library defaults. Poor trials were stopped early by median pruning, which compares a trial’s validation J with the median of earlier trials *at the same training step*. Each study keeps one best configuration, chosen on the 20 validation instances. That configuration was then retrained with 3 random seeds for 1,000,000 steps offline and 300,000 steps online, and results are the mean and standard deviation over the seeds. A follow-up trained online Options 0, 2 and 3 (PPO) for 1,000,000 steps, and Option 2 for 2,000,000 (Appendices C and D).

Evaluation. Every method, RL and baseline, is evaluated on the same 50 held-out test instances for each setting. RL agents act deterministically using a two-stage rule: idle if the probability of idling is at least the total probability of all job actions, otherwise take the most likely job action. We report mean J over the test instances; for RL this is averaged over the 3 seeds, with the standard deviation across seeds. The heuristics are deterministic, so they were run once, with their standard deviation across instances.

We also record metrics that were not optimised, such as the on-time rate, number of late jobs, maximum tardiness and energy. Any pattern in these is a side effect of optimising J rather than a learned trade-off, so it is interpreted with caution.

Multi-objective experiments. To test whether RL’s relative performance depends on the objective, every method was also scored on composite objectives J_λ (Eq. (4)). The weights $\lambda_T, \lambda_U, \lambda_E$ are set separately for each setting, so that each added term equals J on the best heuristic’s schedule; this prevents any term from dominating merely because of its units. Each weight is then multiplied by 0.25, 0.5, 1, 2 and 4. In addition to the agents trained on J alone, Option 2 and 3 PPO agents were trained directly on several composite objectives (300,000 steps offline with default hyperparameters, 3 seeds; 1,000,000 steps online).

4 Results

All results below are from the evaluation on the 50 held-out instances; lower J is better. Full tables for every setting and metric are on the project’s GitHub [results page](#), and an AI-made interactive atlas (a collection of all results in an HTML page) is [here](#).

4.1 RL against Heuristics on the Main Objective

Table 1 and Fig. 2 summarise the main comparison. Looking at the offline case, with weighted jobs the best RL agent (Option 2, PPO) reaches $J = 28,061$, 5.2% above the best heuristic, WLST (26,670). RL’s standard deviation across seeds is small. When every job has the same weight, the problem is much easier for heuristics. LST scores 13,098, and the

best RL agents either reproduce it exactly by simply choosing that heuristic (Option 1), or come within 0.8% (Option 2).

For the online case, RL was only trained at $\rho = 0.95$, and every RL design was far behind the best heuristic: EDF+Consolidate reached 21,986, and the best RL agent (Option 1, A2C), which can choose that same heuristic (i.e. we would expect it to match the best heuristic’s score exactly), got 37,812 (+72%). This online gap could be a training-budget issue.

Table 1: Mean J on 50 test instances (lower is better). RL: mean $\pm$ standard deviation over 3 seeds, best algorithm per design; heuristics are deterministic. Online RL is shown at the standard 300k-step budget.

Method	Offline, weighted	Offline, $w_j = 1$	Online, $\rho = 0.95$
Best heuristic	26,670 (WLST)	13,098 (LST)	21,986 (EDF+Cons.)
LST+FirstFit	39,536	13,098	38,831
EDF+FirstFit	41,302	13,757	40,381
CP-SAT (60 s)	35,681	16,469	–
Option 0 (job–machine pairs)	29,696 $\pm$ 833	13,494 $\pm$ 276	44,920 $\pm$ 0
Option 1 (rule selection)	38,645 $\pm$ 142	13,098 $\pm$ 0	37,812 $\pm$ 8,893
Option 2 (priority)	28,061 $\pm$ 219	13,207 $\pm$ 28	39,947 $\pm$ 11,949
Option 3 (priority + ATC)	28,282 $\pm$ 537	13,496 $\pm$ 98	44,827 $\pm$ 2,890
Option 4 (action branching)	28,577 $\pm$ 43	13,411 $\pm$ 181	139,721 $\pm$ 101,396
Best RL vs best heuristic	+5.2%	0.0%	+72%

4.2 Combined Objectives

Figure 3 answers the entire research question in one view. For each objective and setting, it compares the best-performing RL agent with the best heuristic. Offline with weighted jobs is the only setting where we see RL win. Note that the multipliers indicate the weight of the added term relative to a reference value. At $\times 1$, the added term is worth as much as J on the best heuristic’s schedule; at $\times 4$, it is worth four times as much, so it dominates the objective.

4.3 Online Training Budget

Whilst training multiple RL models, it was found that most of the online gap, especially for Option 2, would fall as training steps increased. Option 2’s mean J falls from 39,947 at 300k steps to 26,515 at 1M (+21% over EDF+Consolidate) and 24,921 at 2M (+13%, 2 seeds), and its best 1M seed is within 4.3% of the heuristic. Option 0 improves on average but remains 55% behind at 1M. Online A2C did not learn: Option 0 A2C produced the same schedule as FCFS+FirstFit ($J = 44,920$) on all three seeds.

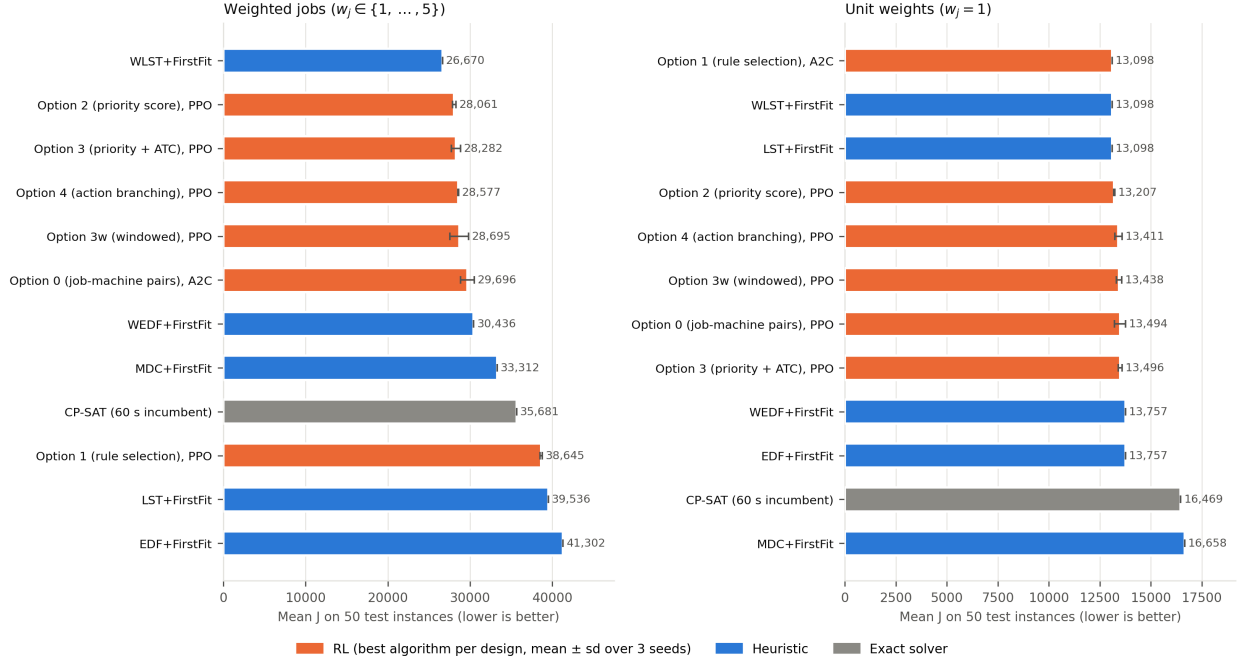


Figure 2: Offline results: the best final agent of each design (orange, mean $\pm$ s.d. over 3 seeds) against the leading heuristics (blue) and CP-SAT’s 60-second schedule (grey). Left: weighted jobs; right: unit weights.

	Offline, weighted jobs	Offline, unit weights	Online, weighted jobs ($\rho = 0.95$)
J only	+5.2% heuristic	+0.0% tie	+13.3% heuristic
J + linear tardiness ($\times 1$)	-1.8% RL	+0.0% tie	+14.9% heuristic
J + linear tardiness ($\times 4$)	-10.3% RL	+0.0% tie	+14.4% heuristic
J + late jobs ($\times 1$)	-10.0% RL	+1.2% heuristic	+14.4% heuristic
J + late jobs ($\times 4$)	-1.4% RL	+10.8% heuristic	+11.5% heuristic
J + energy ($\times 1$)	+4.1% heuristic	+0.7% heuristic	+8.2% heuristic
All four terms ($\times 1$)	-8.6% RL	+2.1% heuristic	+11.3% heuristic

Best RL agent vs best heuristic on each objective. Negative (blue) = RL better; positive (red) = heuristic better.

Figure 3: Best RL agent against the best heuristic on each objective (rows) and setting (columns); blue means RL is better, red means the heuristic is better. Weights are multiples of the reference values, which make each added term equal to J on the best heuristic’s schedule. Only RL agents with at least two evaluated seeds are included.

4.4 Other Findings

Action-space design. Learning with job priority (Options 2 and 3) worked best in every setting. Picking job-machine pairs directly (Option 0) was 6% worse offline and remained 55% behind online at 1M steps. Choosing job and machine with two heads (Option 4) was competitive offline, but failed online. Rule selection (Option 1) would mostly collapse onto a single rule, often reproducing LST exactly.

Training stability. In 8 independent offline runs of the same Option 2 configuration (default hyperparameters, 300k steps), one diverged ($J = 69,367$, against 28,352–30,925 for the other seven). Online seeds differed much more: Option 2 at 300k steps ranged from 30,076 to 53,231.

Energy. Interestingly, energy was the only metric on which RL never beat the best heuristic in any scenario. Training with the energy term had no diverging runs, but it did not make training meaningfully more stable either; it only traded J for lower energy.

No single heuristic wins. The best rule often changes depending on deadline tightness and load: EDF with loose offline deadlines, WLST with medium and tight deadlines, LST with unit weights, and Consolidate placement was the best at every online load.

Hyperparameter tuning. The library defaults were the best configuration in 8 of the 24 RL studies.

5 Discussion

When does RL beat heuristics? Our results suggest three conclusions. First, *the objective must not be one that a simple rule solves well*. RL never improved on the best heuristic in simple cases (Option 1 can only reproduce a rule): with unit weights and squared tardiness, LST matched every proven optimum of the CP-SAT exact solver on small instances. When the objective mixes metrics, no single heuristic is designed for that combination, and this is where we saw our only RL wins. Heuristics are designed for a specific problem and solve it efficiently and effectively. Second, *RL only outperforms when jobs have weights*. Weights give a learned priority more to work with; notably, online only considered weighted jobs due to a lack of training time. Future RL work could combine different metrics into learned job weights to simplify the agent’s task. Third, *the agent must be trained enough for the scenario*. Offline had the most data due to its much faster training times, and 1M steps offline came within 5% of the best heuristic. Online, 300k steps left RL 70% behind, and 1–2M steps slowly closed the gap for the best designs.

Multi-objective results. At moderate weights, the agents that performed best on the composite objectives were, somewhat surprisingly, those trained solely on squared tardiness. Their schedules simply happened to produce fewer late jobs and lower linear tardiness,

making this behaviour a by-product of how the RL policy structures schedules rather than evidence of targeted learning. This cautions against over-interpreting improvements in metrics that were not explicitly optimised. One would normally expect agents trained with explicit composite objectives to do best on those same objectives, yet this did not occur, possibly because they were trained for only 300,000 steps with default hyperparameters, against 1M tuned steps for the J -only agents. At higher weights, however, the agents trained on the composite objective began to dominate, indicating genuine learning aligned with the intended optimisation.

Online difficulty. Online results depend heavily on the random sequence of job arrivals, which the agent cannot predict, so the training signal is much noisier than offline [58]. To reduce this noise, the critic (but not the policy) was given a summary of jobs that had not yet arrived. Because only the critic sees this information, the policy’s learning signal stays unbiased [58, 59], but online RL still remained well behind the best heuristic. Online episodes are also about ten times longer than offline, and the agent must observe over a thousand job slots, so training was slower and far less stable. Several other changes were tested without closing the gap, including potential-based reward shaping [45], a capacity look-ahead, reward scaling and undiscounted returns ($\gamma = 1$). The one change that clearly helped was more training.

Practical considerations. RL is costly to build and to train. Training was unstable, and a single online tuning trial took four to five hours. Beyond compute, RL is expensive to model: reaching the current results took around three months of iterating on the reward, action space, observation and training, and RL still does not beat the best heuristic in most settings. Heuristics, by contrast, need no training, give the same schedule every time, are cheap to run, and are easy to explain. For a cloud provider, a well-chosen heuristic is therefore the practical default, and RL is worth its cost mainly where the objective combines several goals, jobs differ in importance, and enough compute is available.

Limitations. All instances are synthetic and of one size (100 jobs, 10 machines, 4 resources), and results use 3 seeds. Real traces should be examined to get concrete results, along with more training.

6 Conclusion

This project formulated cloud resource allocation as a multi-resource scheduling problem, built offline and online environments, and compared six RL action-space designs, trained with PPO and A2C, against 49 heuristics and an exact solver. On the main objective, weighted squared tardiness, RL came within 5% of the best heuristic offline. When considering multiple objectives, RL did beat the best heuristic by 2–10% for a few metric combinations, but notably the best RL models for some of these combinations were those trained without the added terms in their objective. Online, RL didn’t manage to beat the best heuristic; however, the data indicates it could be a computational issue: as training steps increased, so did RL performance, but larger models and robust testing of them did not fit within the

project timeline. Learning a job priority was the most effective design, although getting RL to train reliably required very careful diagnosis and refinement. RL could be a competitive algorithm for cloud resource allocation, especially with multiple objectives, but this report found that a simple heuristic remains the best choice, especially considering computational and modelling resources.

7 Future Work

The following topics were only touched on briefly in this report and are directions for future research.

- **The online case.** RL was trained online at a single load ($\rho = 0.95$) with weighted jobs. Future work should train for longer and with more seeds, cover the other loads and a unit-weight online setting, and test generalisation across loads and deadlines (in progress).
- **Idling and capacity reservation.** Agents were free to idle, but whether they actually learned to reserve capacity for future jobs was not analysed.
- **Training stability.** The causes of divergence and seed variance, and larger tuning budgets, were not studied in depth.
- **Action spaces and architectures.** Why rule selection collapsed and the two-headed design failed online; graph neural network encoders [60], large discrete action-space methods [61], branching [48] and hybrid architectures [17]; and learned placement to close the energy gap.
- **Alternative formulations.** RCPO and PPO-Lagrangian under the final objective, and smaller formulation choices such as the offline limit of one job start per time step.
- **Objectives and metrics.** Composite-objective agents with the full training setup (in progress), a single agent conditioned on the objective weights, and other metrics such as response time.
- **Realism and scale.** Real traces such as the Google cluster trace [43] or Azure [44], and larger clusters.
- **Heuristic discovery.** Extracting interpretable rules from the learned priorities.

Data Management Plan

All code, raw results (per-instance metrics for every method and seed), processed tables and figures, Optuna study logs and the chronological experiment log are in the public GitHub repository². Results are under **Results/** (raw runs, tuning studies, and the processed **ALL_RESULTS** archive), and every number in this report can be regenerated from them

²<https://github.com/Ethan-Ty-Riekert/Neural-Knapsack-Research-Project>

with the scripts provided. All instances are synthetic and generated from fixed random seeds, so no personal or sensitive data is involved. Trained model files are regenerated by the training scripts and are not version-controlled. The version cited in this report is a tagged release on the `main` branch.

Generative AI Use Statement

Claude Code (Anthropic) was used as a coding and research assistant throughout this project. The original environment and the translation of the mathematical formulation into code were done by hand. Claude Code was used in the following ways:

- **Implementation and experiments.** Implementing models, training and evaluation code, and running experiments from a queue. Every result was reviewed personally.
- **Literature search.** Finding relevant papers and resources. Every source cited in this report was checked by the author.
- **Modelling discussions.** Discussing modelling ideas and design choices against the literature, and giving a second opinion on decisions such as reward design, action spaces and evaluation protocols. Every modelling decision was made and approved by the author.
- **Diagnostics.** Investigating unexpected results and training failures, and checking them against the literature.
- **Report review and editing.** Reviewing the report for consistency with the code and the latest results, checking numbers, claims and citations, and helping to edit the text. All content was reviewed and approved by the author.

Claude worked under a fixed set of rules: every design decision had to be cited and justified, or derived; implementation and discussion had to remain mathematically grounded; and every decision and result had to be recorded in an experiment log. Separate Claude agents were also used weekly to check the code’s efficiency, mathematical rigour and factual accuracy. Using Claude allowed far more experiments, analyses and model iterations than would otherwise have been possible.

References

- [1] K. Saidi and D. Bardou, “Task scheduling and vm placement to resource allocation in cloud computing: Challenges and opportunities,” *Cluster Computing*, vol. 26, no. 5, pp. 3069–3087, 2023. [Online]. Available: www.proquest.com/docview/2918267010
- [2] A. Mohamed, M. Hamdan, S. Khan, A. Abdelaziz, S. F. Babiker, M. Imran, and M. N. Marsono, “Software-defined networks for resource allocation in cloud computing: A survey,” *Computer Networks*, vol. 195, p. 108151, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S1389128621002152>
- [3] V. V. Vinothina, R. Sridaran, and P. Ganapathi, “A survey on resource allocation strategies in cloud computing,” *International Journal of Advanced Computer Science and Applications*, 2012. [Online]. Available: <https://doi.org/10.14569/IJACSA.2012>
- [4] S. M. Parikh, “A survey on cloud computing resource allocation techniques,” in *2013 Nirma University International Conference on Engineering (NUiCONE)*. IEEE, 2013, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/NUiCONE.2013.6780076>
- [5] M. H. Mohamaddiah, A. Abdullah, S. Subramaniam, and M. Hussin, “A survey on resource allocation and monitoring in cloud computing,” *International Journal of Machine Learning and Computing*, vol. 4, no. 1, pp. 31–38, 2014. [Online]. Available: <https://doi.org/10.7763/IJMLC.2014.V4.382>
- [6] N. Devi, S. Dalal, K. Solanki *et al.*, “A systematic literature review for load balancing and task scheduling techniques in cloud computing,” *Artificial Intelligence Review*, vol. 57, p. 276, 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s10462-024-10925-w>
- [7] B. Wang, Z. Lin, W. Kong, and H. Dong, “Bin packing optimization via deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 3, pp. 2542–2549, 2025.
- [8] N. Dahmani, A. Nazir, I. Taleb, and S. M. S. Bukhari, “Reinforcement learning based intelligent optimisation for bin packing problems: A review,” *Array*, vol. 28, p. 100616, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590005625002437>
- [9] S. Alharthi, A. Alshamsi, A. Alseiari, and A. Alwarafy, “Auto-scaling techniques in cloud computing: Issues and research directions,” *Sensors*, vol. 24, no. 17, p. 5551, 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/17/5551>
- [10] S. Bhatnagar and R. Mahant, “Dynamic resource allocation in cloud computing environments: Ai-driven approaches for optimizing workload distribution and cost efficiency,” in *Proceedings of International Conference on Paradigms of Communication, Computing and Data Analytics (PCCDA 2025)*, ser. Algorithms for Intelligent Systems, S. J. Nanda, H. Mittal, and M. H. Lim, Eds. Singapore: Springer, 2026. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-96-6843-4_23

- [11] H. Technologies, *Introduction to Cloud Computing Computing*. New York: Springer, 2022. [Online]. Available: <https://link.springer.com/book/10.1007/978-981-19-3026-3>
- [12] Q. Zhang, L. Cheng, and R. Boutaba, “Cloud computing: State-of-the-art and research challenges,” *Journal of Internet Services and Applications*, vol. 1, pp. 7–18, 2010. [Online]. Available: <https://link.springer.com/article/10.1007/s13174-010-0007-6>
- [13] Z. Y. Lee and X. Tang, “Evaluation of dynamic vector bin packing for virtual machine placement,” in *arXiv preprint*, 2026. [Online]. Available: <https://arxiv.org/html/2602.14704v1>
- [14] V. Anuradha and D. Sumathi, “A survey on resource allocation strategies in cloud computing,” *International Conference on Information Communication and Embedded Systems (ICICES2014)*, pp. 1–7, 2014.
- [15] R. Makkar and N. Mangla, “Load balancing and task scheduling methods in cloud computing – a review,” in *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*. IEEE, 2024, pp. 1037–1043.
- [16] R. Alkhalifa, F. Alkhomayes, B. Almazroua, D. Alhaidan, M. Alothman, and J. Almuhaideb, “A comparative review of parallel exact, heuristic, metaheuristic, and hybrid optimization techniques for the traveling salesman problem,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.18278>
- [17] A. Rathee and S. Dalal, “A systematic literature review of machine learning-based resource allocation techniques in cloud computing,” *Computing Archives for Informatics and Numerical Computation*, vol. 107, no. 9, p. 179, 09 2025. [Online]. Available: <https://www.proquest.com/scholarly-journals/systematic-literature-review-machine-learning/docview/3239203131/se-2>
- [18] J. Zhang, B. Zi, and X. Ge, “Attend2pack: Bin packing through deep reinforcement learning with attention,” *arXiv preprint arXiv:2107.04333*, 2021.
- [19] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.
- [20] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, 2019.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [22] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2016.

- [23] ApX Machine Learning, “A2c and a3c algorithms,” <https://apxml.com/courses/advanced-reinforcement-learning/chapter-3-advanced-policy-gradients-actor-critic/a2c-a3c>, 2026.
- [24] A. Greeks, “Types of actor-critic algorithms in reinforcement learning,” <https://aigreeks.com/types-of-actor-critic-algorithms-in-reinforcement/>, 2024.
- [25] M.-A. Chadia and H. Mousannifa, “Understanding reinforcement learning algorithms: The progress from basic q-learning to proximal policy optimization,” *arXiv preprint arXiv:2304.00026*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.00026>
- [26] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems*, vol. 12. MIT Press, 1999, pp. 1008–1014. [Online]. Available: <https://proceedings.neurips.cc/paper/1999/file/6449f44a102fde848669bdd9eb6b76fa-Paper.pdf>
- [27] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, “Hyper-heuristics: A survey of the state of the art,” *Journal of the Operational Research Society*, vol. 64, no. 12, pp. 1695–1724, 2013.
- [28] S. Lassoued, A. Gobachew, S. Lier, and A. Schwung, “Policy-based deep reinforcement learning hyperheuristics for job-shop scheduling problems,” *arXiv preprint arXiv:2601.11189*, 2026.
- [29] N. Marques, G. Figueira, and L. Guimarães, “Dynamic dispatching rule selection for the job shop scheduling problem,” *Computers and Industrial Engineering*, vol. 210, p. 111471, 2025.
- [30] B. Min and C. O. Kim, “State-dependent parameter tuning of the apparent tardiness cost dispatching rule using deep reinforcement learning,” *IEEE Access*, vol. 10, pp. 20 187–20 198, 2022.
- [31] X. Chen, R. Bai, R. Qu, J. Dong, and Y. Jin, “Deep reinforcement learning assisted genetic programming ensemble hyper-heuristics for dynamic scheduling of container port trucks,” *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 4, 2024.
- [32] A. P. J. Vepsalainen and T. E. Morton, “Priority rules for job shops with weighted tardiness costs,” *Management Science*, vol. 33, no. 8, pp. 1035–1047, 1987.
- [33] L. Perron and V. Furnon, “Or-tools,” Google. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver
- [34] R. Anayat, “Cloud-based reinforcement learning in resource-constrained environments: Real-time performance optimization in autonomous systems,” 11 2024.
- [35] W. Yang, A. Acuto, Y. Zhou, and D. Wojtczak, “A survey for deep reinforcement learning based network intrusion detection,” *Applied AI Letters*, vol. 7, no. 2, p. 22, Jun 01 2026. [Online]. Available: <https://www.proquest.com/scholarly-journals/survey-deep-reinforcement-learning-based-network/docview/3314625396/se-2>

- [36] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” in *Proceedings of the International FLAIRS Conference*, 2020.
- [37] P. Ladosz, L. Weng, M. Kim, and H. Oh, “Exploration in deep reinforcement learning: A survey,” *Information Fusion*, vol. 85, pp. 1–22, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253522000288>
- [38] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, May 1992. [Online]. Available: <https://doi.org/10.1007/BF00992696>
- [39] J. Skalse, N. Howe, D. Krashennnikov, and D. Krueger, “Defining and characterizing reward hacking,” *arXiv preprint arXiv:2209.13085*, 2022.
- [40] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167739X11000689?via%3Dihub>
- [41] X. Fan, W.-D. Weber, and L. A. Barroso, “Power provisioning for a warehouse-sized computer,” in *Proceedings of the 34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, pp. 13–23. [Online]. Available: <https://www.barroso.org/publications/PowerProvisioning.pdf>
- [42] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
- [43] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2012.
- [44] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [45] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *International Conference on Machine Learning (ICML)*, 1999.
- [46] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [47] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *International Conference on Learning Representations (ICLR)*, 2019.
- [48] A. Tavakoli, F. Pardo, and P. Kormushev, “Action branching architectures for deep reinforcement learning,” *arXiv, Tech. Rep. 1711.08946*, 2018.

- [49] W. E. Smith, “Various optimizers for single-stage production,” *Naval Research Logistics Quarterly*, vol. 3, no. 1-2, pp. 59–66, 1956.
- [50] J. J. Kanet and X. Li, “A weighted modified due date rule for sequencing to minimize weighted tardiness,” *Journal of Scheduling*, vol. 7, no. 4, pp. 261–276, 2004.
- [51] D. C. Carroll, “Heuristic sequencing of single and multiple component jobs,” Ph.D. dissertation, Sloan School of Management, Massachusetts Institute of Technology, 1965.
- [52] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, “Multi-resource packing for cluster schedulers,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 455–466, 2014.
- [53] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of ICNN’95 – International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948.
- [54] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [55] C. N. Potts and L. N. Van Wassenhove, “A branch and bound algorithm for the total weighted tardiness problem,” *Operations Research*, vol. 33, no. 2, pp. 363–377, 1985.
- [56] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.
- [57] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems 24 (NeurIPS 2011)*, 2011, pp. 2546–2554.
- [58] H. Mao, S. B. Venkatakrisnan, M. Schwarzkopf, and M. Alizadeh, “Variance reduction for reinforcement learning in input-driven environments,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [59] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, “Asymmetric actor critic for image-based robot learning,” in *Robotics: Science and Systems (RSS)*, 2018.
- [60] “Graph neural networks for job shop scheduling problems: A survey,” *arXiv preprint arXiv:2406.14096*, 2024.
- [61] G. Dulac-Arnold, R. Evans, H. van Hasselt, P. Sunehag, T. Lillicrap, J. Hunt, T. Mann, T. Weber, T. Degris, and B. Coppin, “Deep reinforcement learning in large discrete action spaces,” *arXiv, Tech. Rep. 1512.07679*, 2015.
- [62] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, “What matters for on-policy deep actor-critic methods? a large-scale study,” in *International Conference on Learning Representations (ICLR)*, 2021.

- [63] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, “Implementation matters in deep policy gradients: A case study on PPO and TRPO,” in *International Conference on Learning Representations (ICLR)*, 2020.

A Nomenclature

Table 2: Nomenclature

Sets	
J	Set of jobs, indexed by j
J_t	Jobs waiting (arrived, not yet started) at time t
P_t	Jobs processing (started, not finished) at time t
M	Set of machines, indexed by m
R	Set of resource types, indexed by r
$\mathcal{T}$	Set of discrete time periods, indexed by t
Parameters	
p_j	Processing duration of job j
a_{jr}	Requirement of job j for resource r
C_r	Capacity of each machine for resource r
H	Length of the planning horizon
d_j	Due date of job j
w_j	Priority weight of job j
τ_j	Arrival time of job j (0 offline)
ν, ρ	Arrival rate and load factor (online)
TF, RDD	Tardiness factor and due-date range (offline deadlines)
$\lambda_1, \lambda_2, \lambda_3$	Weights in the initial objective Eq. (3)
$\lambda_T, \lambda_U, \lambda_E$	Weights in the final objective Eq. (4)
Decision variables	
x_{jmt}	1 if job j starts on machine m at time t ; 0 otherwise
y_m	1 if machine m is used; 0 otherwise
s_j	Start time of job j
T_j	Tardiness of job j , $\max(0, s_j + p_j - d_j)$
U_j	1 if job j finishes after d_j ; 0 otherwise
E	Energy: total active machine-time
U_{mrt}	Total usage of resource r on machine m at time t
θ	Peak utilisation ratio, $\max_{m,r,t} U_{mrt}/C_r$

B Constraints

Let $\mathcal{T} = \{0, 1, \dots, \bar{H}\}$, where $\bar{H}$ is any upper bound on the time to finish all jobs (e.g. $H + \sum_j p_j$, under the extended horizon). For each job, let $\mathcal{T}_j = \{t \in \mathcal{T} : \tau_j \leq t \leq \bar{H} - p_j\}$ be its feasible start times.

Assignment. Each job starts exactly once, on exactly one machine, and never before it arrives:

$$\sum_{m \in M} \sum_{t \in \mathcal{T}_j} x_{jmt} = 1, \quad \forall j \in J. \quad (10)$$

Offline start rate. Offline, at most one job starts per time step:

$$\sum_{j \in J} \sum_{m \in M} x_{jmt} \leq 1, \quad \forall t \in \mathcal{T}. \quad (11)$$

Machine activation. A job can only be placed on an active machine:

$$x_{jmt} \leq y_m, \quad \forall j \in J, m \in M, t \in \mathcal{T}_j. \quad (12)$$

Start time.

$$s_j = \sum_{m \in M} \sum_{t \in \mathcal{T}_j} t x_{jmt}, \quad \forall j \in J. \quad (13)$$

Resource usage and capacity. A job started at t' occupies its resources during $[t', t' + p_j)$, so the usage at time t counts every job started within the previous p_j steps. Usage may never exceed capacity:

$$U_{mrt} = \sum_{j \in J} \sum_{\substack{t' \in \mathcal{T}_j \\ t - p_j < t' \leq t}} a_{jr} x_{jmt'} \leq C_r, \quad \forall m \in M, r \in R, t \in \mathcal{T}. \quad (14)$$

Tardiness, late jobs and peak utilisation.

$$T_j \geq s_j + p_j - d_j, \quad T_j \geq 0, \quad T_j \leq \bar{H} U_j, \quad \forall j \in J, \quad (15)$$

$$\theta \geq \frac{U_{mrt}}{C_r}, \quad \forall m \in M, r \in R, t \in \mathcal{T}. \quad (16)$$

Because the objectives are minimised and increase with T_j , U_j and θ , these inequalities hold with equality at the optimum. Finally, $x_{jmt}, y_m, U_j \in \{0, 1\}$.

C Hyperparameter Search Space

D Experimental Details

Offline deadlines. Following Potts and Van Wassenhove [55], each d_j is drawn uniformly from $[\bar{C}(1 - \text{TF} - \text{RDD}/2), \bar{C}(1 - \text{TF} + \text{RDD}/2)]$ and is at least p_j . Here $\bar{C}$ is a lower bound on the time to finish all jobs (the larger of the number of jobs, since one job starts per step, and the total work divided by the cluster's capacity), TF is the tardiness factor and $\text{RDD} = 0.6$ the range of due dates, which sets how spread out deadlines are.

Table 3: Hyperparameters searched by Optuna (ranges after [62]). “log” means sampled log-uniformly; lists are categorical choices. Trial 0 of every study used the library defaults instead.

Hyperparameter	PPO	A2C
Learning rate	$3 \times 10^{-5} - 10^{-3}$ (log)	$10^{-4} - 3 \times 10^{-3}$ (log)
Rollout length (all 4 environments)	2048, 4096, 8192, 16384	20, 80, 320
Minibatch size	64, 256, 1024	–
Epochs per update	3, 5, 10	–
Discount factor γ	0.99, 0.995, 0.999	0.99, 0.995, 0.999
GAE λ	0.9, 0.95, 0.98	0.9, 0.95, 1.0
Clip range	0.1, 0.2, 0.3	–
Entropy coefficient	$10^{-4} - 3 \times 10^{-2}$ (log)	$10^{-4} - 3 \times 10^{-2}$ (log)

Online deadlines and job sizes. A job’s deadline is its arrival time plus a slack drawn uniformly from $\{10, \dots, 60\}$ time steps ($\{5, \dots, 25\}$ in a tight-deadline variant). Durations and demands are log-normal, not Gaussian: the logarithm of the size is normally distributed (with $\sigma = 1$), giving many small jobs and a few large ones. The location parameter is set so that the mean equals that of the offline uniform distribution, and sizes are capped at four times the offline maximum (durations of at most 40 steps).

Tuning protocol. Each of the 24 studies ran 12 trials of 300,000 steps with 4 parallel environments; trial 0 used the Stable-Baselines3 defaults, which lie outside the log-scaled search space (the default entropy coefficient is 0). Every 75,000 steps, a trial reported its mean J on 5 validation instances. Optuna’s median pruner stopped the trial if this was worse than the median of the values earlier trials had reported at the same step; pruning only starts after 5 trials have finished. A completed trial’s score is its mean J on all 20 validation instances, and the best score gives the study’s single chosen configuration. The offline configuration was reused for the unit-weight setting. Three online PPO studies (Options 0, 2 and 4) trained at about 20 steps per second, roughly 4–5 hours per trial, and were stopped after 3–7 trials to meet the project deadline; the best completed trial was used as usual. The defaults were the best configuration in 8 of the 24 studies.

Faults fixed in the final training version. Each fault was measured directly in training logs.

1. *The critic did not learn.* Squared-tardiness returns are in the hundreds to thousands, and the value and policy losses share one clipped gradient, so the value error swamped each update. The critic explained none of the variance in returns, and A2C policies stayed close to uniformly random. Rewards are now divided by a running estimate of the standard deviation of the return [63, 62]; afterwards the critic explained 30–99.9% of the variance.
2. *Deterministic evaluation idled.* Idle is a single action, while the probability of starting a job is split across many (job, machine) actions. For an undecided policy, idle could

be the single most likely action in every state, so a “most likely action” rule idled indefinitely even though the trained (sampling) policy placed jobs 98% of the time. The two-stage rule of Section 3.6.1 fixes this, and is identical to the most-likely-action rule for confident policies.

3. *Idling could repeat forever.* An idle used to advance time by one step and return the agent to an unchanged decision. Event-driven idling advances to the next arrival or completion instead. Waiting between events never helps, since a job that fits at an intermediate step also fits now and finishes earlier, so no schedule is lost.

Together, these changes reduced offline Option 0 A2C’s validation J from 234,116 to 43,164.